

RSA Signatures Under Hardware Restrictions

Marc Joye
NXP Semiconductors



Yan Michalevsky
Anjuna Security



Motivation

- We want to compute an RSA signature
 - Use a Hardware Security Module (HSM)
 - Use particular parameters (i.e. a specific public exponent)
- But... the HSM does not support those parameters



Intel SGX Enclave Signing

- Intel implemented secure enclaves as part of the Software Guard Extensions (SGX)
- Initially restricted which vendors could execute production enclaves by requiring signing the enclave binary
- Enclaves have to be signed with a 3072-bit RSA signature, using public exponent $e = 3$

RSA Signature

$\text{KeyGen}(k)$: **Output a public key** $pk = \{N, e\}$

and a secret key $sk = \{d\}$ **such that** $e \cdot d \equiv 1 \pmod{\varphi(N)}$

$\text{Sign}(N, d, m)$: $m \rightarrow \mu(m)$ $\sigma = \mu(m)^d \pmod{N}$

$\text{Verify}(N, e, m, \sigma)$: **if** $\mu(m) \equiv \sigma^e \pmod{N}$ **output** true
 otherwise, output false

Small public exponent e results in a faster verification

Low-exponent attacks

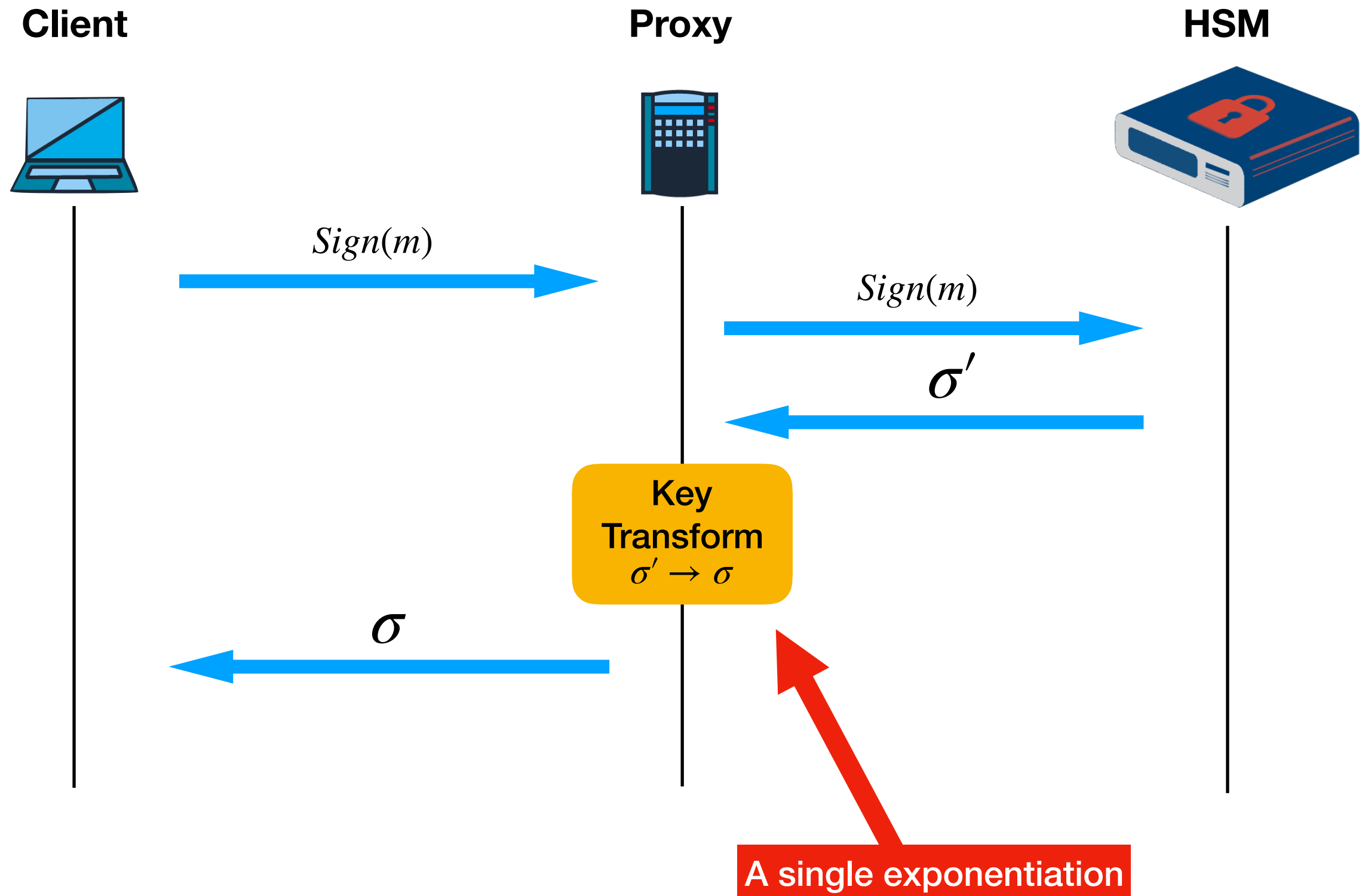
- Low-*private* exponent - an actual problem
- Low-public exponent - attacks are prevented by proper padding schemes
 - Attacks due to Coppersmith's theorem
 - Håstad's attack
 - Franklin et al. - attacks on encryption under the same modulus

“While there are no conclusive proofs of its unsuitability, the use of public exponent $e = 3$ is discouraged, especially when used in conjunction with padding schemes like PKCS#1 v1.5. We chose not to add support for other exponents than $F_4 = 2^{16} + 1$ which is the de-facto industry standard.”

— Yubico (regarding YubiHSMv2)



Scheme



Key Transform

Setup

- Pick a *random* exponent
 $t \in \mathbb{Z}$ **such that** $0 < t < N$ **and** $\gcd(t, \varphi(N)) = 1$
- Compute $e' = e \cdot t \pmod{\varphi(N)}$ and check that e' is supported by the HSM
- Compute $d' = (e')^{-1} \pmod{\varphi(N)}$ and store it in the HSM
- (Dispose of any copies of e' and d' outside the HSM)

Note: It is mandatory that e' not be made public

Key Transform

- Given a message m the HSM generates a signature $\sigma' = \mu(m)^{d'} \pmod{N}$
- We convert σ' to a signature verifiable using e by computing

$$\sigma = (\sigma')^t \pmod{N}$$

The public knowledge of t does not compromise the security of the scheme

Importance of e' secrecy

Key-recovery attack using the Extended GCD

If $\gcd(e, e') = 1$ the Extended Euclidean algorithm yields a, b such that

$$a \cdot e + b \cdot e' = 1$$

Since $e' = e \cdot t \pmod{\varphi(N)}$ then

$$1 \equiv a \cdot e + b \cdot e \cdot t \equiv e(a + b \cdot t) \pmod{\varphi(N)}$$

and the adversary can compute

$$D = a + b \cdot t \equiv e^{-1} \equiv d \pmod{\varphi(N)}$$

that enables the adversary to produce a valid signature (and factor N)

Security

- The scheme is EUF-CMA secure (the strongest security notion for signatures), under the RSA assumption in the random-oracle model
 - For concreteness, we instantiate with RSA-FDH
- Proof of security using a simulator, with a small probability of failure resulting in a gap in the security reduction
- The gap in our security reduction amounts to less than 7-bits of security, meaning the construction is secure for practical applications

Final notes

- QVRSA is used for faster verification of the RSA signature by the Intel SGX Launch Enclave. Our scheme is compatible with QVRSA since it is completely oblivious to the transform we perform.
- Our scheme readily extends to additional RSA-based signature schemes (RSA-PSS, Katz and Wang). Similarly there would be an additional factor in the security bound.



www.anjuna.io

cs.stanford.edu/~yanm2

joye.site88.net